

1. Backend Architecture and Technology Stack

The Number Plate Detection system's backend is developed using Spring Boot with Java, implementing the Model-View-Controller (MVC) architectural pattern. The system facilitates seamless communication between frontend and backend components through REST APIs, which serve as the primary entry points for all client requests. When users capture or upload vehicle images, the data is transmitted to the backend via a dedicated upload API. The controller layer subsequently validates incoming requests before delegating them to the service layer for business logic processing.

2. REST API and Controller Layer Design

The Spring Boot framework exposes RESTful API endpoints that handle all incoming requests from the frontend interface. The Controller layer acts as the initial gateway, responsible for receiving uploaded images, performing request validation, and directing validated requests to the service layer.

Client (Frontend) → POST /upload → Controller → Service Layer

Validation Checks Performed by Controller:

Prior to forwarding requests to the service layer, the controller executes the following validation procedures:

File format validation to ensure compatibility with supported image types

File size verification to enforce upload limits

Request parameter validation to confirm all required fields are present and correctly formatted

3. Business Logic Implementation — Service Layer

The Service layer encapsulates the core business logic of the Number Plate Detection system. Upon receiving a validated image request from the controller, the service layer orchestrates a sequential workflow of operations to process the image and generate detection results.

Sequential Processing Workflow:

- Image Validation — Perform comprehensive validation of the uploaded image, including integrity checks and format verification
- Metadata Generation — Create associated metadata including timestamps, unique filename generation, and request tracking information
- System Health Verification — Confirm connectivity and availability of dependent services, including FTP server connection and database accessibility

- **Image Transfer** — Securely transmit the validated image to the configured FTP server for storage and processing
- **Model Invocation** — Trigger the number plate detection algorithm to process the image and extract license plate information
- **Result Persistence** — Store detection results, system logs, and processing metadata in the database for future reference and analysis

4. Centralized Storage Architecture — FTP Server

Following successful validation, the uploaded image is transferred to a centralized FTP server for persistent storage. This architectural decision effectively offloads storage responsibilities from the application server, ensuring optimal performance while maintaining secure and accessible image repositories for subsequent processing by the detection model.

Service Layer → FTP Connection Establishment → Image Upload → Centralized Storage

Image Metadata Tracking:

For each uploaded image, the system maintains the following associated metadata in the database:

1. **Image filename** — Unique identifier for the stored image
2. **Upload timestamp** — Date and time of successful image transfer
3. **FTP file path** — Complete directory path for image retrieval and reference

5. ML Model Integration with OCR

Upon successful image storage on the FTP server, the detection pipeline is automatically triggered. The machine learning model accesses the stored image, processes it to identify and isolate the number plate region, and generates a precise bounding box around the detected plate area. Subsequently, Optical Character Recognition (OCR) technology is applied to extract alphanumeric characters from the identified region, converting the visual plate information into machine-readable text.

Load Image from FTP → ML Model Inference → Bounding Box Generation → OCR Processing → Extracted Plate Text

6. Data Persistence Layer — PostgreSQL Database

All detection results and system-generated logs are persistently stored in a PostgreSQL relational database. The backend leverages Hibernate ORM (Object-Relational Mapping) to streamline database interactions and execute SQL operations efficiently. For every processed request, the system records comprehensive information in the database schema.

Database Schema Structure:

Field	Description
filename	Name of the uploaded image file
upload_time	Timestamp of the image upload
detected_plate	Extracted alphanumeric text from the number plate
processing_status	Current state of processing (Success/Failure)
server_logs	System activity logs for auditing and debugging

7. System Observability — Logging and Monitoring Framework

A comprehensive logging infrastructure is implemented to track and monitor activities across all stages of the processing pipeline. System logs are generated for critical events, facilitating effective debugging, performance monitoring, and operational insights.

Monitored Events and Logging Points:

1. API Request Logging — Timestamp and details of incoming API requests
2. Storage Confirmation — Successful image transfer to FTP server with metadata
3. Model Trigger Events — Initiation of detection pipeline with tracking identifiers
4. Processing Outcomes — Final detection status (Completed/Failed) with result summaries
5. Error Tracking — Detailed error messages and stack traces for troubleshooting and root cause analysis

8. Architectural Summary and System Capabilities

The Number Plate Detection backend is architected as a scalable, high-performance system that seamlessly integrates multiple technology components: Spring Boot for application logic, FTP server for distributed image storage, PostgreSQL for persistent data logging, and a machine learning model for intelligent plate detection. This modular architecture ensures optimal resource utilization and system reliability.

Key Architectural Advantages:

- Concurrent Request Handling — Leverages Spring Boot's built-in multithreading capabilities to process multiple user requests simultaneously without performance degradation
- Scalable Deployment — Packaged as a self-contained JAR file, enabling straightforward deployment across various environments, from development to production
- Modular Integration — Decoupled components (storage, processing, database) allow for independent scaling and maintenance
- Real-time Processing — Optimized pipeline ensures rapid image processing and detection results for real-time vehicle monitoring applications